

Circuit Breaker

 systemdesignschool.io/fundamentals/circuit-breaker



Retries handle transient failures by attempting requests multiple times. But what happens when a service fails persistently? Retrying a completely down service wastes resources and adds latency. Circuit breakers detect these persistent failures and stop making doomed requests.

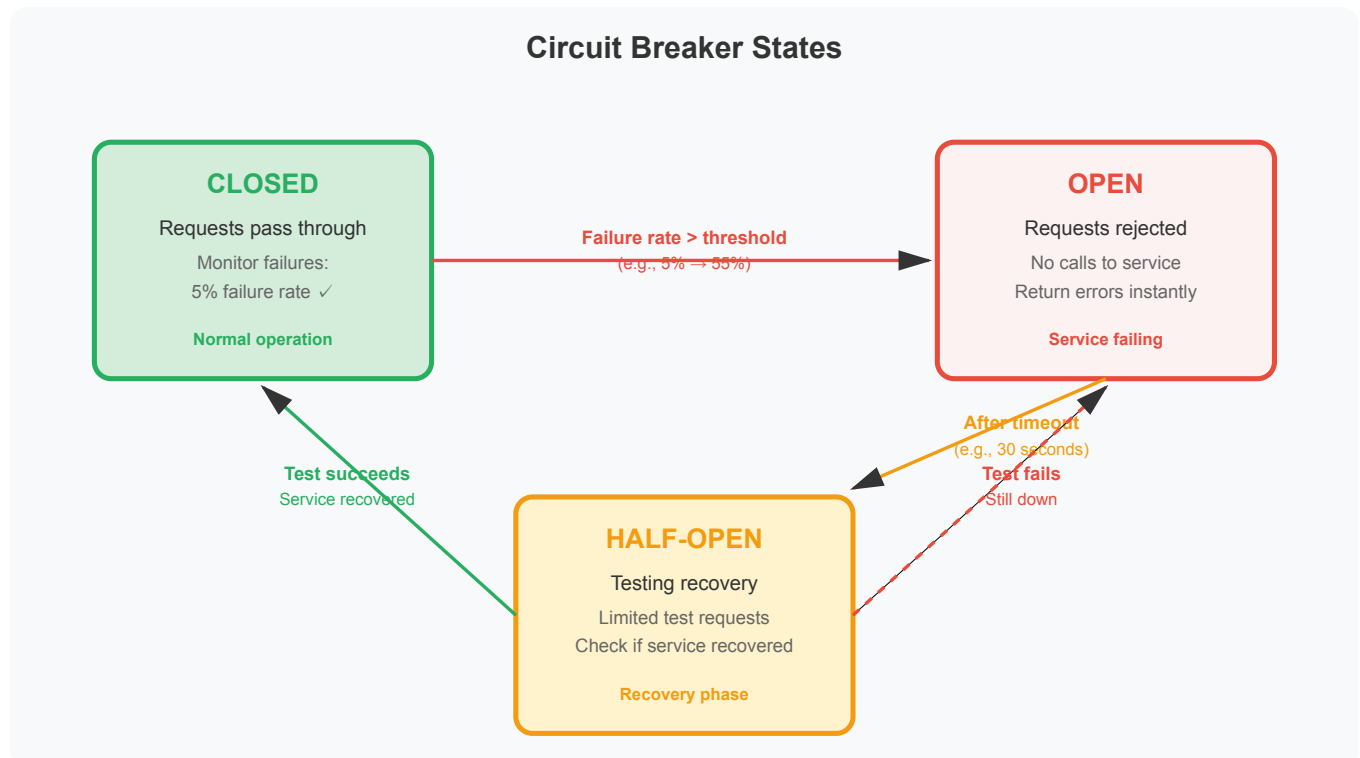
The Physical Analogy



An electrical circuit breaker protects your home from overload. When an appliance draws too much current, the breaker trips and cuts power. This prevents damage to wiring and other devices. After you fix the problem, you can manually reset the breaker to restore power.

A microservices circuit breaker works similarly. When a service fails repeatedly, the circuit breaker "trips" and stops sending requests. This protects upstream services from wasting resources on calls that will fail. After a timeout, the circuit breaker tests if the service recovered and reopens if it did.

Three States



In the **closed** state, the circuit breaker allows all requests through. It monitors each request and tracks failures. If 10 out of 100 requests fail, the failure rate is 10%. The circuit remains closed as long as the failure rate stays below the threshold.

When failures exceed the threshold, the circuit transitions to the **open** state. A service normally has a 5% failure rate. Suddenly it climbs to 60%. The circuit breaker detects this and opens. In the open state, the circuit breaker immediately rejects all requests without attempting to call the service. This prevents wasting threads and network resources on calls that will almost certainly fail.

After a timeout period, the circuit transitions to **half-open**. The circuit breaker allows a small number of test requests through. If these requests succeed, the service has recovered. The circuit closes and normal traffic resumes. If the test requests fail, the service is still down. The circuit reopens for another timeout period.

Configuration Parameters

The failure threshold determines when the circuit opens. A threshold of 50% means the circuit opens when half of requests fail. Set this based on normal failure rates. If a service normally fails 5% of requests, a 30% threshold gives enough headroom.

The timeout period controls how long the circuit stays open before testing recovery. A 30-second timeout means after the circuit opens, it waits 30 seconds before allowing test requests. Too short and you overwhelm a service trying to recover. Too long and users experience degraded service unnecessarily.

The window size affects how failures are counted. A rolling window of 100 requests means the circuit tracks the last 100 requests. If 50 of those 100 failed, the failure rate is 50%. This window should be large enough to smooth out noise but small enough to detect problems quickly.

Implementation

Libraries like Hystrix and Resilience4j provide circuit breaker implementations. In Python with pybreaker:

```
from pybreaker import CircuitBreaker breaker = CircuitBreaker(fail_max=5,
timeout_duration=30) @breaker def call_inventory_service(): response =
requests.get("http://inventory/stock/123", timeout=2) return response.json()
try: inventory = call_inventory_service() except CircuitBreakerError: # Circuit
is open, use fallback return {"stock": 0, "source": "default"}
```

Monitoring Circuit State

Track circuit breaker state changes in metrics. Count how many times each circuit opens. Log when circuits transition states. Dashboard these metrics so operators can see which services are having problems.

A circuit that opens frequently indicates a dependency problem. Either the downstream service is unreliable or your failure threshold is too aggressive. A circuit that stays open for extended periods means a service is completely down and needs attention.

Working with Fallbacks

When a circuit opens, requests fail immediately. Return a fallback response instead of an error. If the recommendations circuit opens, show popular items. If the user profile circuit opens, return cached data. The [fallbacks](#) section covers graceful degradation strategies in detail.

Circuit breakers and fallbacks together create resilient systems. The circuit breaker detects persistent failures quickly. The fallback provides acceptable alternatives. Users experience degraded functionality instead of complete failure.